DATA ENGINEERING & CLOUD · LESSON 2 OF 15

Ingestion: Batch, Streaming, API, File, Database and Change Data Capture

Move data from its source into the platform reliably. Compare batch, streaming, API pulls, file drops, database extracts and change data capture (CDC), and learn the patterns that make each one safe to rerun, including watermarks, paging and deduplication.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Choose an ingestion method from freshness, volume and source capabilities
- Ingest a file and an API into a landing zone with a load record
- Use watermarks and pagination to fetch only new data
- Deduplicate events and explain what CDC captures

Datasets used

`daily_sales.csv`

What ingestion is

Ingestion copies data from a source into the platform's landing zone without changing its meaning. The landing zone is the first place the data lives on your side: a bucket, a folder or a raw table. Keeping an unchanged copy makes every later step repeatable, because you can rebuild the rest from it.

CASE STUDY **Three sources, three ways in**

A retailer needs sales from its point-of-sale files (delivered as daily CSVs), its online store (a REST API with new orders) and its inventory database (changes must appear within the hour). The same platform needs three ingestion methods. Choosing wrongly means either stale data or a source overloaded by constant queries.

The five methods

Method	How it works	Freshness	Load on source	Typical use
Batch	Copy everything, or a period, at intervals	Hours to days	Periodic and heavy	Daily files, full table dumps
Streaming	Events are sent as they happen	Seconds	Low per event	Clicks, sensor data, payments
API pull	Call an endpoint, page through results	Minutes to hours	Depends on rate limits	SaaS tools, partner data
File drop	A system places files in a folder or bucket	Per drop	Low	Vendor exports, reports
Database extract	Query a table for rows changed since a point	Minutes to hours	Query load on the database	Operational tables
Change data capture (CDC)	Read the database's change log	Seconds to minutes	Very low	Keeping a warehouse close to live

Incremental pulls with a watermark

Copying everything every time is expensive. An **incremental** pull fetches only rows changed since the last successful run. The last value seen (the **watermark**, often an `updated_at` timestamp) is stored in a control table and advanced only after the load succeeds.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
source = pd.read_csv("daily_sales.csv", parse_dates=["date"]).rename(columns={"date":
"updated_at"})
con.register("source_rows", source)

# Control table: one row per source, holding the last watermark that loaded successfully
con.execute("CREATE TABLE IF NOT EXISTS load_control (source VARCHAR PRIMARY KEY, watermark
TIMESTAMP)")
con.execute("CREATE TABLE IF NOT EXISTS landing_sales (updated_at TIMESTAMP, units_sold
BIGINT)")

def incremental_load(con, source_name, cutoff):
    wm = con.execute("SELECT watermark FROM load_control WHERE source = ?",
[source_name]).fetchone()
    last = wm[0] if wm else pd.Timestamp("1900-01-01")
    new = con.execute("SELECT * FROM source_rows WHERE updated_at > ? AND updated_at <= ?",
                      [last, cutoff]).fetchdf()
    con.register("batch", new)
    con.execute("INSERT INTO landing_sales SELECT updated_at, units_sold FROM batch")
    new_wm = new["updated_at"].max() if len(new) else last
    con.execute("INSERT OR REPLACE INTO load_control VALUES (?, ?)", [source_name, new_wm])
    return len(new), new_wm

print(incremental_load(con, "pos_daily", pd.Timestamp("2025-01-31")))
print(incremental_load(con, "pos_daily", pd.Timestamp("2025-01-31")))   # rerun: loads
nothing new
print(con.execute("SELECT COUNT(*) FROM landing_sales").fetchone())
```

OUTPUT

```
(1127, Timestamp('2025-01-31 00:00:00'))
(0, datetime.datetime(2025, 1, 31, 0, 0))
(1127,)
```

The second run loads nothing, because the watermark already covers that period. The watermark moves only when the load succeeds, so a failed run leaves it where it was and the next run retries the same rows. The rerun safety here is the same idea as the idempotent loads in the SQL lessons.

::::warn Watermarks on event time can miss late rows A row dated yesterday that arrives today is skipped by a watermark on the event date. Use the time the record was written or updated when the source provides it, or re-load a short safety window and deduplicate. ::::

Paging through an API

APIs rarely return everything at once. They return pages, often with a cursor or a `next` link. The ingestion code must follow every page, handle rate limits, and stop only when the source says there is nothing left.

Python

```python
# Illustrative pattern (not run: needs a live API). Replace the URL and field names
# with the source's documentation.
import time
import requests

def fetch_all(url, token, page_size=100, max_retries=5):
    headers = {"Authorization": f"Bearer {token}"}
    cursor, rows = None, []
    while True:
        params = {"limit": page_size}
        if cursor:
            params["cursor"] = cursor
        for attempt in range(max_retries):
            resp = requests.get(url, headers=headers, params=params, timeout=30)
            if resp.status_code == 429:              # rate limited: wait, then retry
                time.sleep(2 ** attempt)
                continue
            resp.raise_for_status()
            break
        body = resp.json()
        rows.extend(body["data"])
        cursor = body.get("next_cursor")
        if not cursor:
            return rows                               # no more pages: stop
```

Every API has its own limits and paging style. Read the documentation before writing the loop, and record how many rows each run fetched so that a silent change in page size is visible.

Streaming and deduplication

Streaming sources can deliver the same event twice (for example, after a network retry). Deduplicate on a unique event identifier, not on the whole row, because two genuine events can look identical.

Python

```python
import pandas as pd

events = pd.DataFrame({
    "event_id": ["e1", "e2", "e2", "e3"],          # e2 was delivered twice
    "order_id": ["A10", "A11", "A11", "A12"],
    "amount":   [120.0, 80.0, 80.0, 45.0],
})
unique_events = events.drop_duplicates(subset=["event_id"], keep="last")
print(unique_events)
print("duplicates removed:", len(events) - len(unique_events))
```

OUTPUT

```
  event_id order_id  amount
0       e1      A10   120.0
2       e2      A11    80.0
3       e3      A12    45.0
duplicates removed: 1
```

Change data capture

CDC reads the database's own log of inserts, updates and deletes, instead of querying the tables. It sees every change, including deletes that a query would miss, and it puts little load on the database.

Approach	Sees deletes?	Sees every change?	Load on source
Query by updated_at	No (deleted rows vanish)	Only if the column is reliable	Query load
Full reload	Yes, by comparison	Yes, but slowly	Heavy
CDC from the log	Yes	Yes	Very low

CDC needs access to the database log and careful handling of schema changes: when a column is added, the capture must know about it before the next change arrives. Tools built for this exist; check the documentation of the database and tool you choose.

Landing zone rules

- Store the raw copy unchanged, with the load time and source name attached.
- Keep one load record per run: source, rows read, rows written, watermark, status.
- Never transform data on the way in; transform in the next layer.
- Keep raw data long enough to rebuild the layers above it.

INTERVIEW CORNER

How would you ingest data from a source that has no timestamp column?

Options: a full reload with a comparison of keys to find inserts, updates and deletes, if the table is small; CDC from the database log, if access is available; or a trigger that writes changes to an audit table. State the trade-off: full reloads are simple but heavy, CDC is precise but needs access and maintenance.

::::key What to remember - Pick ingestion by freshness, volume and what the source allows. - Incremental loads use a watermark that moves only after success. - Follow every page of an API and respect its rate limits. - Deduplicate streams on a unique event ID. - CDC captures deletes and changes with little load, but needs access and schema care. ::::

Quick quiz

5 questions · tap an answer to check it

1. When does a watermark move forward?

A At the start of the run

B Only after the load succeeds

C Every hour regardless of success

D When the source changes

Answer: B. A failed run must leave the watermark where it was.

2. Why follow the `next` cursor of an API?

A To make the data bigger

B Because a single response does not contain all the records

C To reduce cost to zero

D To change the schema

Answer: B. Paging is how APIs return large results.

3. What is the main advantage of CDC?

A It is always the cheapest to build

B It captures every change, including deletes, with little load on the source

C It requires no access

D It replaces the warehouse

Answer: B. CDC reads the change log rather than querying tables.

4. Two rows in a stream are identical except for their timestamps. What should you do?

A Remove both

B Deduplicate on the unique event ID, since they may be genuine separate events

C Keep only the first row by content

D Ignore duplicates

Answer: B. Event IDs, not row contents, identify a unique event.

5. A source has no updated_at column and is small. Which approach is simplest and safe?

A Stream it

B A full reload with key comparison to find changes

C Ignore changes

D Store only the first row

Answer: B. A small table can be reloaded and compared without special access.

EXERCISE **Ingest with a safety window**

Using `daily_sales.csv`, write an incremental load that uses a 3-day safety window: each run reloads the last three days of rows and deduplicates on the date. Run it twice with the same cutoff and once with a cutoff two days later, and report the row count after each run.

Show solution (worked answer)

Hints: keep the watermark as the maximum date loaded. Deduplicate on `updated_at` after the insert, keeping the last. The row count should not change on a rerun with the same cutoff, and should grow only by the new days.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.